

1 Implementation of Array operations Insertion, Deletion and Display.

```

#include <iostream>
#include <conio.h> // For Turbo compatibility
using namespace std;
class Array {
    int arr[50], pos, elem, size;
    bool array = true;
public:
    void create() {
        if(array){
            cout << "\nEnter the size of the array: ";
            cin >> size;
            cout << "Enter array elements: ";
            for (int i = 0; i < size; i++)
                cin >> arr[i];
            display();
            array = false;
        }
        else
            cout<<"Array already created...";

    }
    void insert() {
        cout << "\nEnter the position to insert (1 to " << size + 1 << "): ";
        cin >> pos;
        if (pos < 1 || pos > size + 1) {
            cout << "\nInvalid position!";
            return;
        }

        cout << "Enter the element to insert: ";
        cin >> elem;

        for (int i = size - 1; i >= pos - 1; i--)
            arr[i + 1] = arr[i];

        arr[pos - 1] = elem;
        size++;

        cout << "\nArray after insertion: ";
        display();
    }
}

```

```

void remove() {
    cout << "\nEnter the position to delete (1 to " << size << "): ";
    cin >> pos;
    if (pos < 1 || pos > size) {
        cout << "\nInvalid position!";
        return;
    }

    elem = arr[pos - 1];
    for (int i = pos - 1; i < size - 1; i++)
        arr[i] = arr[i + 1];

    size--;
    cout << "\nDeleted element: " << elem;
    cout << "\nArray after deletion: ";
    display();
}

void display() {
    cout << "\nArray elements: ";
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}
};

int main() {
    Array obj;
    int ch;
    // clrscr(); // Turbo support

    do {
        cout << "\n\n***** MENU *****";
        cout << "\n1. Create";
        cout << "\n2. Insert";
        cout << "\n3. Delete";
        cout << "\n4. Exit";
        cout << "\nEnter your choice: ";
        cin >> ch;
        switch (ch) {
            case 1: obj.create(); break;
            case 2: obj.insert(); break;
            case 3: obj.remove(); break;
            case 4: cout << "\nExiting..."; break;
        }
    } while (ch != 4);
}

```

```

        default: cout << "\nInvalid choice! Try again.";
    }
} while (ch != 4);
// getch(); // Turbo
return 0;
}

```

2 Implementation of Stack using array .

```

#include<iostream>
using namespace std;
class Stack{
    int *stk, top, max;
public:
    Stack(){
        top = -1;
        cout<<"\nEnter the length of the stack : ";
        cin>>max;
        stk = new int [max];
    }
    void push();
    void pop();
    void show();

    ~Stack(){
        delete []stk;
    }
};

void Stack::pop(){
    int t;
    if(top== -1)
        cout<<"\nstack is empty";
    else
        t = stk[top];
        cout<<"\nelemnt "<<t<<"is removed.";
        top--;
}

void Stack::push(){
    int item;
    if(top == max-1){
        cout<<"\nStack is overflow ";
    }
}

```

```
        else{
            cout<<"\n enter the item to be inserted : ";
            cin>>item;
            top++;
            stk[top] = item;
        }
    }

void Stack::show(){
    if(top == -1){
        cout<<"\nstack is empty";
        return;
    }
    else{
        for(int i = top;i>=0; i--){
            cout<<"\t"<<stk[i];
        }
    }
}

int main () {
    Stack obj;
    int ch;
    do{
        cout<<"\n1.Push\n2.Pop\n3.Show\n4.Exit : ";
        cin>>ch;
        switch(ch){
            case 1: obj.push(); break;
            case 2: obj.pop(); break;
            case 3: obj.show(); break;
            case 4: break;
            default : cout<<"\n invalid choice";
        }
    }while(ch!=4);

    return 0;
}
```

6 Implementation of Linear Search.

```

#include<iostream>
using namespace std;

class Linear{
    int arr[10],num;
    public:
        Linear(){
            cout<<"Enter the number of element : ";
            cin>>num;
            cout<<"Enter "<<num<<" elements : ";
            for(int i=0;i<num;i++){
                cin>>arr[i];
            }
            cout<<"\nArray is : \n";
            for(int i=0;i<num;i++){
                cout<<"\t"<<arr[i];
            }
        }
        int search(int item){
            for(int i=0;i<5;i++){
                if(arr[i]==item){
                    return i;
                }
            }
            return -1;
        }
};

int main(){
    Linear obj;
    int elem,loc = -1;
    cout<<"\nEnter the element to search : ";
    cin>>elem;
    loc = obj.search(elem);
    if(loc == -1)
        cout<<"not found";
    else
        cout<<"found at location "<<loc;
}

```

7 Implementation of Binary Search.

```
#include <iostream>
using namespace std;

class Binary
{
private:
    int size;
    int *arr;

public:
    void getData()
    {
        cout << "Enter the size of array ";
        cin >> size;
        arr = new int[size];
        cout << "Enter array elements : ";
        for (int i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }
    void putData()
    {
        cout<<"\n";
        for (int i = 0; i < size; i++)
        {
            cout << arr[i] << "\t";
        }
    }

    int Search()
    {
        int target , index = -1;
        cout << "\nEnter the target : ";
        cin >> target;

        int s = 0, e = size-1;
        while (s<=e)
        {
            int mid = (s+e)/2;
            if(arr[mid] == target){
                return mid;
            }else if (arr[mid]>target)
```

```
        {
            e =mid-1;
        }else
            s=mid+1;
    }return -1;

}

};

int main()
{
    Binary obj;
    obj.getData();
    obj.putData();
    int result = obj.Search();
    if (result == -1)
    {
        cout << "Element not found";
    }
    else
    {
        cout << "Element found at index :"<<result;
    }

    obj.putData();
}
```

8 Implementation of Bubble sort.

```
#include <iostream>
using namespace std;

class Bubble
{
private:
    int size;
    int *arr;

public:
    void sortArray()
    {
        for (int i = 0; i < size - 1; i++)
        {
            for (int j = 0; j < size - i - 1; j++)
            {
                if (arr[j] > arr[j + 1])
                {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }

    void getData()
    {
        cout << "Enter the size of array ";
        cin >> size;
        arr = new int[size];
        cout << "Enter array elements : ";
        for (int i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }

    void putData()
    {
        cout << "\n";
        for (int i = 0; i < size; i++)
        {
            cout << arr[i] << "\t";
        }
    }
}
```



```
    }  
  }  
};  
  
int main()  
{  
    Bubble obj;  
    obj.getData();  
    cout << "Before Sort : ";  
    obj.putData();  
    obj.sortArray();  
    cout << "\nAfter Sort : ";  
    obj.putData();  
    return 0;  
}
```

9 Implementation of Selection sort.

```
#include <iostream>
using namespace std;

class Selection
{
private:
    int size;
    int *arr;

public:
    void sortArray()
    {
        for (int i = 0; i < size - 1; i++)
        {
            int small = i;
            for (int j = i+1; j < size; j++)
            {
                if (arr[small] > arr[j])
                {
                    small = j;
                }
            }

            if(i!=small){
                int temp = arr[i];
                arr[i]=arr[small];
                arr[small] = temp;
            }
        }
    }

    void getData()
    {
        cout << "Enter the size of array ";
        cin >> size;
        arr = new int[size];
        cout << "Enter array elements : ";
        for (int i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }

    void putData()
```

```
{
    cout << "\n";
    for (int i = 0; i < size; i++)
    {
        cout << arr[i] << "\t";
    }
}
};
```

```
int main()
{
    Selection obj;
    obj.getData();
    cout << "Before Sort : ";
    obj.putData();
    obj.sortArray();
    cout << "\nAfter Sort : ";
    obj.putData();
    return 0;
}
```

10 Implementation of Insertion sort.

```

#include <iostream>
using namespace std;

class Insertion
{
private:
    int size;
    int *arr;

public:
    void sortArray()
    {
        for (int i = 1; i < size; i++)
        {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key)
            {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }

    void getData()
    {
        cout << "Enter the size of array ";
        cin >> size;
        arr = new int[size];
        cout << "Enter array elements : ";
        for (int i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }

    void putData()
    {
        cout << "\n";
        for (int i = 0; i < size; i++)
        {
            cout << arr[i] << "\t";
        }
    }
}

```

```
    }  
};  
  
int main()  
{  
    Insertion obj;  
    obj.getData();  
    cout << "Before Sort : ";  
    obj.putData();  
    obj.sortArray();  
    cout << "\nAfter Sort : ";  
    obj.putData();  
    return 0;  
}
```

11 Implementation of Quick sort.

```
#include <iostream>
using namespace std;

class QuickSort {
    int *arr;
    int n;

    void swap(int &a, int &b) {
        int temp = a;
        a = b;
        b = temp;
    }

    int partition(int low, int high) {
        int pivot = arr[high];
        int i = low - 1;

        for (int j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;
                swap(arr[i], arr[j]);
            }
        }
        swap(arr[i + 1], arr[high]);
        return i + 1;
    }

public:
    QuickSort() {
        cout << "Enter the size of the array: ";
        cin >> n;
        arr = new int[n];
        cout << "Enter " << n << " elements: ";
        for (int i = 0; i < n; i++)
            cin >> arr[i];
    }

    void display() {
        for (int i = 0; i < n; i++)
            cout << arr[i] << " ";
        cout << endl;
    }
}
```

```
void sort(int low, int high) {
    if (low < high) {
        int pi = partition(low, high);
        sort(low, pi - 1);
        sort(pi + 1, high);
    }
}

void performSort() {
    cout<<"\n Before Sorting : \n";
    display();
    sort(0, n - 1);
    cout<<"\n After Sorting : \n";
    display();
}

~QuickSort() {
    delete[] arr;
}

};

int main() {
    QuickSort q;
    q.performSort();
    return 0;
}
```

12 Implementation of Merge sort.

```

#include <iostream>
using namespace std;

class MergeSort {
    int *A;
    int n;

    void merge(int low, int mid, int high) {
        int *B = new int[n];
        int i = low, j = mid + 1, k = low;

        while (i <= mid && j <= high) {
            if (A[i] <= A[j])
                B[k++] = A[i++];
            else
                B[k++] = A[j++];
        }
        while (i <= mid) B[k++] = A[i++];
        while (j <= high) B[k++] = A[j++];

        for (int t = low; t <= high; t++)
            A[t] = B[t];

        delete[] B;
    }

    void mergeSort(int low, int high) {
        if (low < high) {
            int mid = (low + high) / 2;
            mergeSort(low, mid);
            mergeSort(mid + 1, high);
            merge(low, mid, high);
        }
    }

public:
    MergeSort(int size) {
        n = size;
        A = new int[n];
    }

    void getData() {
        cout << "Enter " << n << " elements: ";
    }

```



```
        for (int i = 0; i < n; i++)
            cin >> A[i];
    }

    void display(const string &msg) {
        cout << msg;
        for (int i = 0; i < n; i++)
            cout << A[i] << " ";
        cout << endl;
    }

    void sortArray() {
        mergeSort(0, n - 1);
    }

    ~MergeSort() {
        delete[] A;
    }
};

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    MergeSort m(n);
    m.getData();
    m.display("\nBefore Sorting: ");
    m.sortArray();
    m.display("After Sorting: ");

    return 0;
}
```

13 Implementation of Binary Search Tree.

```

#include <iostream>
#include <conio.h>
using namespace std;

class Node {
public:
    int info;
    Node *left, *right;
};

class BST {
    Node *root;
public:
    BST() {
        root = NULL;
    }

    void create();
    void search();
    void deletion();
    void display(Node *ptr, int level);
    Node* getRoot() { return root; }
};

void BST::create() {
    char ch;
    do {
        Node *n = new Node;
        cout << "\nEnter the element: ";
        cin >> n->info;
        n->left = n->right = NULL;

        if (root == NULL)
            root = n;
        else {
            Node *t = root, *p;
            while (t != NULL) {
                p = t;
                if (n->info < t->info)
                    t = t->left;
                else
                    t = t->right;
            }
        }
    }
}

```

```

        if (n->info < p->info)
            p->left = n;
        else
            p->right = n;
    }

    cout << "\nAdd another (Y/y): ";
    cin >> ch;
} while (ch == 'Y' || ch == 'y');
}

void BST::search() {
    int item, level = 1;
    cout << "\nEnter item to search: ";
    cin >> item;

    Node *p = root;
    while (p != NULL && item != p->info) {
        if (item < p->info)
            p = p->left;
        else
            p = p->right;
        level++;
    }

    if (p == NULL)
        cout << "\nItem not found.";
    else
        cout << "\nItem found at level: " << level;
}

void BST::deletion() {
    int item;
    cout << "\nEnter item to delete: ";
    cin >> item;

    Node *p = root, *pp = NULL;
    while (p != NULL && p->info != item) {
        pp = p;
        if (item < p->info)
            p = p->left;
        else
            p = p->right;
    }
}

```

```

if (p == NULL) {
    cout << "\nItem not found.";
    return;
}

cout << "\nDeleted item: " << p->info;

// Case 1 & 2: Node with 0 or 1 child
Node *c;
if (p->left == NULL)
    c = p->right;
else if (p->right == NULL)
    c = p->left;
else {
    // Case 3: Node with 2 children
    Node *s = p->left, *ps = p;
    while (s->right != NULL) {
        ps = s;
        s = s->right;
    }
    p->info = s->info;
    if (ps == p)
        ps->left = s->left;
    else
        ps->right = s->left;
    delete s;
    return;
}

if (pp == NULL)
    root = c;
else if (p == pp->left)
    pp->left = c;
else
    pp->right = c;

delete p;
}

void BST::display(Node *ptr, int level) {
    if (ptr != NULL) {
        display(ptr->right, level + 1);
        cout << "\n";
    }
}

```

```

        if (ptr == root)
            cout << "Root->: ";
        else
            for (int i = 0; i < level; i++)
                cout << "    ";
        cout << ptr->info;
        display(ptr->left, level + 1);
    }
}

int main() {
    BST b;
    int ch;
    // clrscr();

    do {
        cout << "\n***** MENU *****";
        cout << "\n1. Create";
        cout << "\n2. Search";
        cout << "\n3. Deletion";
        cout << "\n4. Display";
        cout << "\n5. Exit";
        cout << "\nEnter your choice: ";
        cin >> ch;

        switch (ch) {
            case 1: b.create(); break;
            case 2: b.search(); break;
            case 3: b.deletion(); break;
            case 4: b.display(b.getRoot(), 1); break;
            case 5: cout << "\nExiting..."; break;
            default: cout << "\nInvalid choice!";
        }

        getch();
    //    clrscr();
    } while (ch != 5);

    return 0;
}

```

14 Implementation of In-order, Pre-order and Post-order Traversals.

```

#include <iostream>
using namespace std;

class node {
public:
    int info;
    node *left, *right;
} *root;

class BST {
public:
    BST() { root = NULL; }
    void create();
    void inorder(node *ptr);
    void preorder(node *ptr);
    void postorder(node *ptr);
};

void BST::create() {
    char ch;
    node *n, *t, *p;
    do {
        n = new node;
        cout << "\nEnter The Element: ";
        cin >> n->info;
        n->left = NULL;
        n->right = NULL;

        if (root == NULL)
            root = n;
        else {
            t = root;
            while (t != NULL) {
                p = t;
                if (n->info < t->info)
                    t = t->left;
                else
                    t = t->right;
            }
            if (n->info < p->info)
                p->left = n;
            else
                p->right = n;
        }
    } while (ch != 'q');
}

```

```

    }
    cout << "\nDo you want to add another node (Y/N): ";
    cin >> ch;
} while (ch == 'Y' || ch == 'y');
}

void BST::inorder(node *ptr) {
    if (ptr != NULL) {
        inorder(ptr->left);
        cout << ptr->info << " ";
        inorder(ptr->right);
    }
}

void BST::preorder(node *ptr) {
    if (ptr != NULL) {
        cout << ptr->info << " ";
        preorder(ptr->left);
        preorder(ptr->right);
    }
}

void BST::postorder(node *ptr) {
    if (ptr != NULL) {
        postorder(ptr->left);
        postorder(ptr->right);
        cout << ptr->info << " ";
    }
}

int main() {
    BST b;
    int ch;
    do {
        cout << "\n***** MENU *****";
        cout << "\n1. Create BST";
        cout << "\n2. In-order Traversal";
        cout << "\n3. Pre-order Traversal";
        cout << "\n4. Post-order Traversal";
        cout << "\n5. Exit";
        cout << "\nEnter Your Choice: ";
        cin >> ch;
        switch (ch) {
            case 1: b.create(); break;
            case 2: cout << "\nIn-order Traversal: "; b.inorder(root); break;
            case 3: cout << "\nPre-order Traversal: "; b.preorder(root); break;

```

```

        case 4: cout << "\nPost-order Traversal: "; b.postorder(root); break;
    }
    cout << endl;
} while (ch != 5);
return 0;
}

```

15 Implement the program for graph representation in memory.

```

#include<iostream>
#include<conio.h>
using namespace std;

class Graph {
    int adj[10][10], n;
    char nodes[10];
public:
    void create();
    void display();
};

void Graph::create() {
    cout << "\nEnter number of nodes: ";
    cin >> n;
    cout << "\nEnter node names: ";
    for (int i = 0; i < n; i++)
        cin >> nodes[i];

    cout << "\nEnter adjacency matrix:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> adj[i][j];
        }
    }
}

void Graph::display() {
    cout << "\nAdjacency Matrix Representation:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << adj[i][j] << " ";
        }
        cout << endl;
    }
}

```



```
int main() {  
    // clrscr();  
    Graph g;  
    g.create();  
    g.display();  
    getch();  
}
```